



project 2

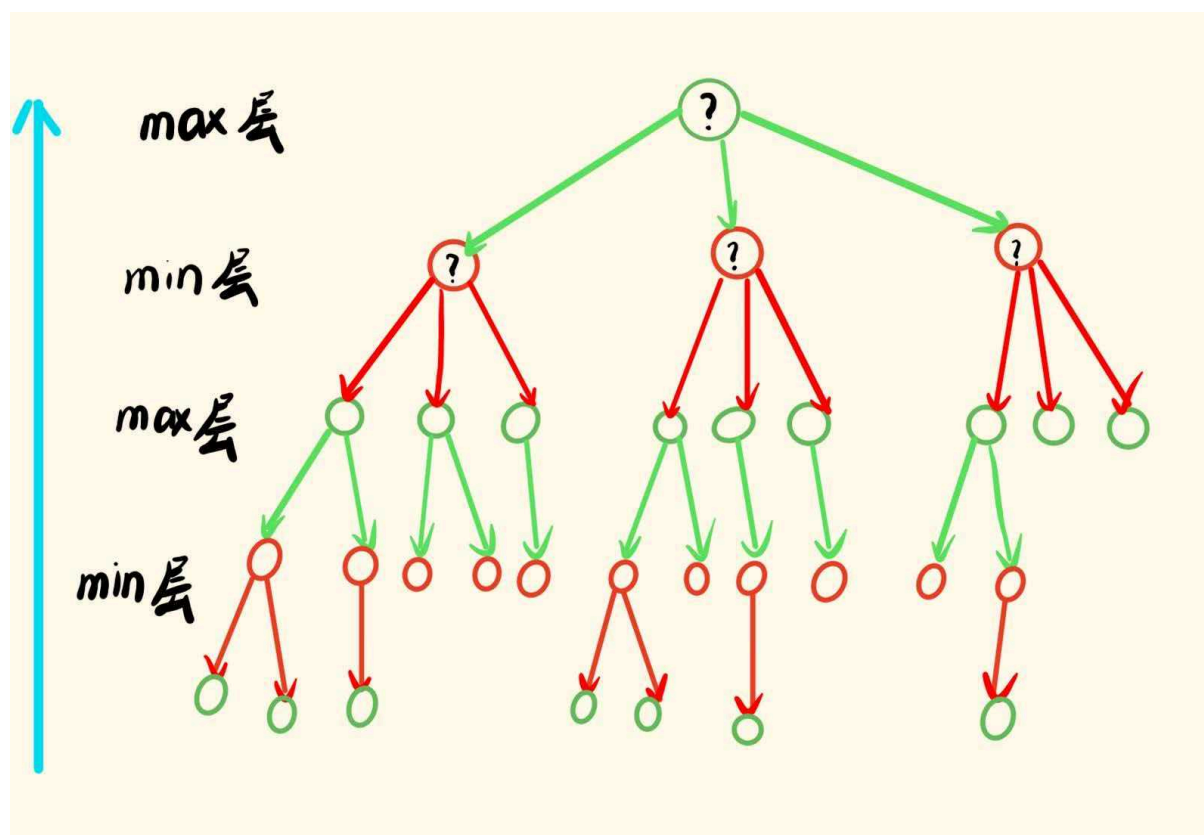
🕒 创建时间 2026年5月8日 14:07

一、背景知识

1. minimax算法

使用场景：动态博弈（你下一步我下一步）

根据每个状态有不同的决策，可以构造出决策树如下：



- 圆圈：局面（状态）
- 绿色：我方
- 红色：对方

- 箭头：做出决策
- ? : 当前局面的分数

选择逻辑：所以首先要定义一下某个状态下获得的分数，然后绿色代表自己，总是要选择导致分数最大的局面，所以绿色会选择子树里面分数最大的。红色代表对手，假设对手是很强的，总会选择对我们最不利的局面，所以红色会选择子树里面分数最小的。

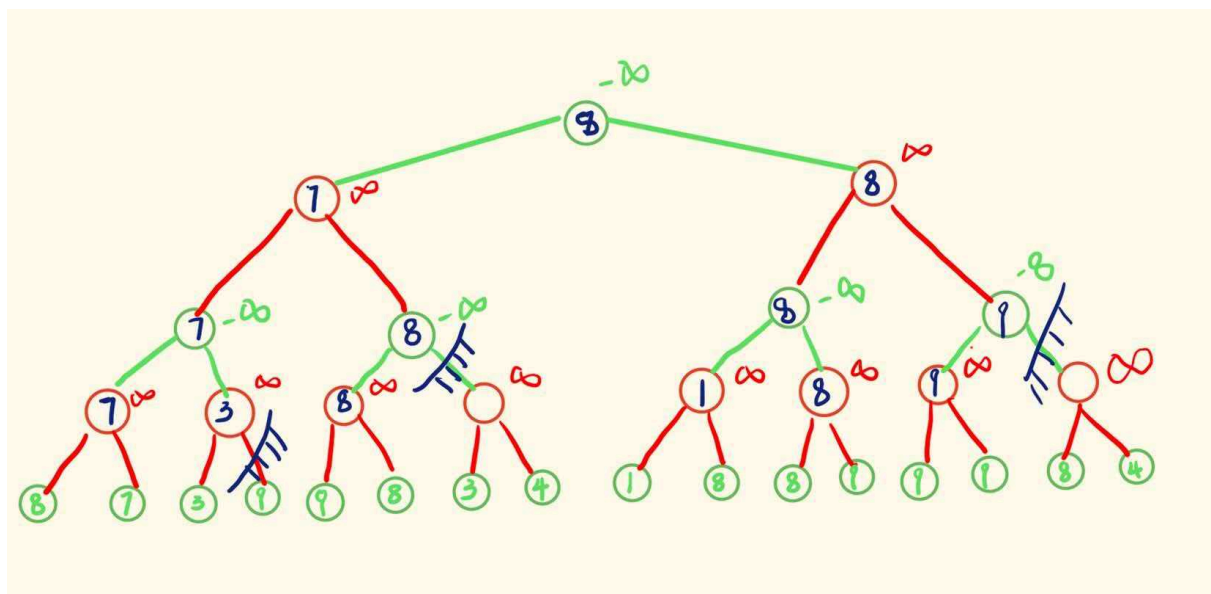
具体实现：从下往上看，先看终局的分数，然后再选择最大/最小，依次往上走，直到走到决策树的根，找到路径。

2. α - β 剪枝

使用场景：在 minimax 算法 的基础上进行优化，用来减少不必要的搜索分支。

核心思想：在决策树搜索过程中，有些分支即使继续算下去，也不可能影响最终选择结果，所以可以提前停止搜索，这个过程叫做剪枝。

- α (alpha)：当前路径上，绿色节点，也就是我方已经能保证拿到的最好分数，是我方的下界：我至少可以拿到这么多。
- β (beta)：当前路径上，红色节点，也就是对方已经能把我们限制到的最低分数，是对方的上界：对方最多只会让我拿到这么多。



剪枝逻辑

- 在绿色节点，也就是我方节点，我方会选择分数最大的子节点。如果当前已经找到一个分数很高的选择，而上一层红色节点不会允许我方获得这么高的分数，那么剩下的兄弟节点就不用看了。也就是：**如果 $\alpha \geq \beta$ ，停止搜索后面的分支。**
- 在红色节点，也就是对方节点，对方会选择让我们分数最小的子节点。如果当前已经找到一个很低的分数，而上一层绿色节点已经有更好的选择，那么这个红色节点剩下的分支也不用看了。同样：**如果 $\alpha \geq \beta$ ，停止搜索后面的分支。**

具体实现：

α - β 剪枝本质上还是从上往下搜索决策树，然后从下往上传递分数。只不过在搜索过程中，会额外带着 α 和 β 两个参数。

初始时：

```
 $\alpha = -\infty$   
 $\beta = +\infty$ 
```

然后：

1. 如果当前是终局，直接返回局面分数
2. 如果当前是我方节点，选择子节点中的最大值，并更新 α
3. 如果当前是对方节点，选择子节点中的最小值，并更新 β
4. 每次更新后判断是否满足 $\alpha \geq \beta$
5. 如果满足，就剪掉剩下还没搜索的分支

二、PJ实现

1. 项目内容

想象你在玩吃豆人（Pacman）。游戏里有：

- 你（Pacman）：要吃光所有豆子
- 幽灵（Ghosts）：要抓住你

你的任务是写一个智能 Pacman，让它自己决定每一步往哪走。这个项目一共有 5 个问题（Q1~Q5），从简单到复杂。

2、游戏的运行机制

回合制

游戏是轮流行动的：

Pacman 走一步 → Ghost1 走一步 → Ghost2 走一步 → ... → 又轮到 Pacman → ...

- agentIndex=0 永远是 Pacman
- agentIndex=1, 2, 3... 是各个幽灵

关键函数

方法	作用
gameState.getLegalActions(agentIndex)	返回某角色当前可以走的方向列表，如 ['North', 'South', 'East', 'Stop']
gameState.generateSuccessor(agentIndex, action)	返回某角色执行某个方向后的新状态
gameState.isWin() / gameState.isLose()	判断当前状态是赢还是输
gameState.getScore()	当前分数
gameState.getNumAgents()	总共几个角色（1个Pacman + N个幽灵）

multiAgents.py

类/函数	问题	作用
ReflexAgent	Q1	只看当前一步做决策
scoreEvaluationFunction	工具	直接返回游戏分数
MultiAgentSearchAgent	基类	提供 self.depth 和 self.evaluationFunction
MinimaxAgent	Q2	向前看N步，假设幽灵最坏
AlphaBetaAgent	Q3	应该优化Minimax的速度
ExpectimaxAgent	Q4	假设幽灵随机走
betterEvaluationFunction	Q5	更聪明的打分函数

3、Q1: ReflexAgent（反射型智能体）

Q1 是最简单的方式：只看眼前。把当前可以走的每个方向都"试想一下"走完什么样，给每种结果打个分，选分最高的走。不往前多想，只看一步。所以考虑因素主要有：距离食物的远近，是否靠近ghost(要分这个ghost害不害怕分情况讨论)，并且要尽量动起来，不能一直禁止。

第一步：试走一下

用 generatePacmanSuccessor(action) 得到"如果走这个方向，会是什么状态"

第二步：提取信息

- newPos：走完后你在哪
- newFood：走完后还剩哪些豆子
- newGhostStates：走完后幽灵在哪、是否害怕

第三步：打分（分数越高 = 这个方向越好）

- ① 如果你选了 STOP（不动），直接给 -500 分（几乎禁止不动）
 - ② 基础分 = 游戏当前分数
 - ③ 食物分：离最近的食物越近，加分越多（鼓励去吃豆子）
 - ④ 幽灵分：
 - 如果幽灵是害怕状态：离得越近加分越多（鼓励追着害怕的幽灵去吃）
 - 如果幽灵正常：离得太近扣分（惩罚危险行为）
-

4、Q2：MinimaxAgent（极小化极大）

Q1 的问题：只看一步。比如前面有个豆子，但走过去幽灵就在旁边——Q1 看不到这个危险。

Q2 的解决办法：往前多想几步。把未来 Pacman 和幽灵的交替行动都模拟出来，形成一棵“博弈树”

Q2 的核心假设：幽灵是最聪明的对手，它们每一步都会选对 Pacman 最不利的方向。

- Pacman 的回合（agentIndex=0）：选所有选项中得分最高的那个 → MAX 节点
- 幽灵的回合（agentIndex>=1）：选所有选项中得分最低的那个 → MIN 节点

一个重要概念：depth（搜索深度）

不希望这个搜索一直算到游戏结束，那样太慢了。self.depth 限制“往前看几轮”。

关键规则：depth 只在“一轮结束”时才减 1。一轮 = Pacman 走一次 + 所有幽灵各走一次。

5、Q3：AlphaBetaAgent（ α - β 剪枝）

Q2 的问题：如果每种情况都算到结尾，会浪费，详情见背景知识

Q3 的解决办法：新增两个变量 α 和 β ，进行判断是否为多余的情况，从而节约资源

核心：当 $\alpha \geq \beta$ 时会停止继续在当前分支检索。

6、Q4：ExpectimaxAgent（期望）

Q3的问题：每次都假设对手最聪明，对我们最不利，但是实际情况往往不是这样

Q4的解决办法：取对手行为会导致结果的平均值，而不是对我们最有害的一个，从而考虑我们的决策。

核心：把Q2的代码min层改为取平均。

7、Q5：betterEvaluationFunction（更好的评估函数）

综合**四个特征**评估局面：

- 输赢直接返回极端值
 - 到最近食物的倒数距离（越近越好）
 - 剩余胶囊数量惩罚（越少越好，鼓励吃胶囊）
 - 幽灵距离：害怕幽灵越近越好，正常幽灵越远越好
-

三、反思总结

1. 博弈树是 AI 决策的基础框架。任何轮流决策的对抗场景（棋类、游戏、谈判）都能用这个框架建模。
2. 搜索和评估是互补的。搜索深度不够时靠评估函数弥补，评估粗糙时靠更深搜索弥补。Minimax + 简单评估 = 浅层搜索 + 好评估，两种思路殊途同归。
3. 剪枝不是偷懒，是智慧。 α - β 剪枝教会我：算得多不等于算得好，知道"什么不需要算"和知道"怎么算"同样重要。
4. 模型假设决定行为。Minimax 的"最坏假设"让 Pacman 过于保守，Expectimax 的"随机假设"让 Pacman 敢于冒险。选什么模型取决于对真实对手的了解。
5. 特征工程是 AI 的手艺活。选对特征、调好权重，比堆砌复杂算法更有效。

首先就是要找到一个合适的评估函数，给每个状态打分。

如果要保守一点就用minimax算法，假设对方都能找到最优解，在最坏的情况下找到最优路径，由于已知对方是最优解，因此可以跳过一些无关痛痒的情况，就是 α - β 剪枝。

如果想要稍微激进一点，就用ExpectimaxAgent，算对手的期望，也就是平均值，可能会获得更大的分数。但是要注意，因为要算期望，所以每一种情况都要算，不能省略或者跳过。